

第1章：基礎

変数・型・関数 — Rust の基本的な構成要素

1 変数 let

let キーワードで変数を宣言する。Rust は型推論を持つので多くの場合型を省略できる。命名規則はスネークケース (snake_case)。

```
1 fn main() {
2     let x = 5;           // 型推論 → i32
3     let y: i64 = 10;     // 型を明示
4
5     // シャドウイング：同じ名前で再宣言できる
6     let x = x + 1;       // x は 6 (新しい変数)
7     let x = "hello";     // 型が変わっても OK
8 }
```

Q. スネークケースってなんですか？

単語と単語をアンダースコア _ でつなぐ命名規則。my_variable のように横に這うのがヘビ (snake) っぽいことからそう呼ばれる。アンダースコアがなくてもコンパイルはできるが、キャメルケース (myVariable) を使うと警告が出る。コンパイラ自体が警告を出すレベルの、公式が推奨する慣習。

2 変数の変更 mut

変数はデフォルトで不変 (immutable)。変更したいときは mut を付ける。

```
1 fn main() {
2     let x = 5;
3     // x = 6; ← コンパイルエラー！ 不変なので変更不可
4
5     let mut y = 5;
6     y = 6;           // OK
7 }
```

3 基本的な型

```
1 let x = 12;           // デフォルトでは i32
2 let a = 12u8;         // サフィックスで型を直接指定
3 let b = 4.3;          // デフォルトでは f64
4 let c = 4.3f32;
5 let bv = true;        // bool
6 let t = (13, false);  // タプル (i32, bool)
7 let sentence = "hello world!"; // &str
```

整数型

型	符号	ビット幅	値の範囲（概略）
u8	なし	8 bit	0 ~ 255
u32	なし	32 bit	0 ~ 約 4.3×10^9
u64	なし	64 bit	0 ~ 約 1.8×10^{19}
u128	なし	128 bit	0 ~ 約 3.4×10^{38}
i8	あり	8 bit	-128 ~ 127
i32	あり	32 bit	約 $\pm 2.1 \times 10^9$
i64	あり	64 bit	約 $\pm 9.2 \times 10^{18}$
i128	あり	128 bit	約 $\pm 1.7 \times 10^{38}$
usize	なし	arch 依存	ポインタサイズ（配列インデックス用）
isize	あり	arch 依存	ポインタサイズ（符号付き）

Q. ポインタサイズ整数型とは？

プログラムが動く環境のビット幅に合わせたサイズ。64ビット PC では `usize = 64bit`。配列のインデックスは必ず `usize` でないとコンパイルエラーになる。「配列のインデックス = `usize`」と覚えておけば OK。

タプル

異なる型の値をひとまとめにしたもの。配列は「全部同じ型」だが、タプルは「型がバラバラでも OK」。

	型	長さ
配列 <code>[i32; 3]</code>	全部同じ	固定
タプル <code>(i32, f64, bool)</code>	バラバラで OK	固定

Q. タプル型ってなんですか？

「名前・年齢・身長」のように型が違う値をまとめたいときに使う。要素へのアクセスは `result.0`、`result.1` のように書く。主な使い道は関数から複数の値を返すとき。もっと複雑になったら後の章で出てくる構造体（`struct`）を使うのが Rust らしいやり方。

4 基本型の変換 `as`

Rust は型に厳格で、異なる数値型を混在させると（計算の中で型が違うと）コンパイルエラーになる。`as` キーワードで変換する。

```
1 fn main() {
2     let a: i32 = 13;
3     let b = a as f64;      // i32 → f64 (13.0)
4
5     let c: f64 = 3.99;
6     let d = c as i32;      // f64 → i32 (3) ← 切り捨て（四捨五入
                             // ではない）
7 }
```

5 定数 const

プログラム全体で使う固定値は `const` で定義する。型の明示が必須。命名は `SCREAMING_SNAKE_CASE`。

```
1 const MAX_POINTS: u32 = 100_000;
2
3 fn main() {
4     println!("{}", MAX_POINTS);
5 }
```

Q. インライン展開ってなんですか？

コンパイル時に定数の名前が値に直接置き換えられること。変数はメモリのどこかに値が置かれそこを参照しに行くが、定数はコンパイル時に値が埋め込まれるのでメモリを参照しに行かない。「定数は変数よりちょっと効率がいい」くらいの理解で OK。

6 配列 [T; N]

同じ型の要素を固定長で並べたもの。長さはコンパイル時に確定する。

```
1 fn main() {
2     let nums: [i32; 3] = [1, 2, 3];
3     println!("{}", nums[0]);    // 1 (0始まり)
4     println!("{}", nums.len()); // 3
5
6     let zeros = [0; 5];          // [0, 0, 0, 0, 0]
7
8     // インデックスは usize が必要
9     let i: usize = 1;
10    println!("{}", nums[i]);     // 2
11 }
```

Q. 「長さはコンパイル時に確定する」とは？

定数と同じイメージで、配列の長さ `[i32; 3]` の `3` はコンパイル時に決まっていけない。実行中にユーザーの入力で長さを決めたい場合は、後の章で出てくる `Vec` (可変長配列) を使う。

7 関数 fn

`fn` キーワードで定義。引数には必ず型を書く。戻り値の型は `->` で指定。

```
1 fn add(x: i32, y: i32) -> i32 {
2     x + y // セミコロンなし → この値が戻り値
3 }
4
5 fn main() {
6     let result = add(5, 3);
7     println!("{}", result); // 8
8 }
```

重要：ブロックの最後の式（セミコロンなし）が戻り値になる。`return` も使えるが、最後の式で返すのが Rust らしいスタイル。

8 複数の戻り値

タプルを使って複数の値を返せる。

```
1 fn swap(x: i32, y: i32) -> (i32, i32) {
2     return (y, x);
3 }
4
5 fn main() {
6     let result = swap(123, 321);
7     println!("{}", result.0, result.1);
8
9     // デストラクチャリング：タプルを直接2つの変数に分解して受け取る
10    let (a, b) = swap(result.0, result.1);
11    println!("{}", a, b);
12 }
```

Q. タプルを2つの変数に分解とは？

`let (a, b) = swap(...)` は、タプルの形に合わせて変数を並べると自動で対応する値が入る書き方。「形に合わせて直接代入」するイメージ。普通に `result.0`、`result.1` でアクセスするのと同じ意味だが、すっきり書ける。

9 空の戻り値 ()

戻り値の型を省略すると、関数は `()`（ユニット型、空のタプル）を返す。他言語の `void` に相当するが、Rust では値として扱える。

```
1 // この2つは同じ意味
2 fn say_hello() -> () {
3     println!("こんにちは");
4 }
5
6 fn say_bye() { // -> () を省略
7     println!("さようなら");
8 }
```

その他のメモ

警告 (warning) について

「変数を宣言したけど一度も使っていないよ」という親切な注意。エラーではなくコンパイルは成功している。消したいときは変数名の先頭に `_` をつける (例: `let _x = 12`)。

`{:?}` とは？

デバッグ用の表示フォーマット。通常の `{}` はタプルや配列を表示できないが、`{:?}` なら「とりあえず中身を全部見せて」という指定で表示できる。バグったときに変数の中身を確認するのに便利。